Filling Up: For each cell (i, j), compare the corresponding characters:

If they match: dp[i][j] = dp[i-1][j-1] + 1 (You add 1 because you found a common character, plus the best answer for the remaining parts.)

If they don't match: dp[i][j] = max(dp[i-1][j], dp[i][j-1]) (You take the best answer from either dropping a character from the first string or the second.)

I don't understand how this calculations make sense in matching and non matching case prove it clearly please ith simple example and tracing and illustration

Let's break down the logic of the recurrence step by step using a very simple example. We'll w ork with two short strings:

- **String 1 (S1):** `"ABC"`
- **String 2 (S2):** `"AC"`

The goal is to find the length of the Longest Common Subsequence (LCS) between these two strings using dynamic programming (DP).

Step 1. Setting Up the DP Table

We create a table `dp` where:

- The **rows** represent the characters of S1 plus an extra row for the empty prefix.
- The **columns** represent the characters of S2 plus an extra column for the empty prefix.

The idea is that `dp[i][j]` will store the LCS length for the substrings:

- S1's first **i** characters (i.e., `S1[0...i-1]`) and
- S2's first **j** characters (i.e., `S2[0...j-1]`).

For empty substrings, the LCS is 0. So we initialize the first row and first column with zeros.

Here's the initial table:

	""	A	C
""	0	0	0
A	0		
B	0		
C	0		

Step 2. Filling the Table

We fill the table cell by cell starting from the top-left (beyond the base row and column). The recurrence relations are:

1. **If the characters match:** If `S1[i-1] == S2[j-1]`, then we have found a new common character. So, `dp[i][j] = dp[i-1][j-1] + 1` (We add 1 to include the current matching character.)
2. **If they don't match:** Then we decide which character to "drop" by taking the maximum of:
   - `dp[i-1][j]` (ignoring the current character from S1) and
   - `dp[i][j-1]` (ignoring the current character from S2). So, `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`

Let's fill the table row by row:

Filling Row 1 (for S1's "A"):

- **Cell (1,1):** Compare S1[0] = `"A"` with S2[0] = `"A"`. They **match**, so: `dp[1][1] = dp[0][0] + 1 = 0 + 1 = 1`.
- **Cell (1,2):** Compare S1[0] = `"A"` with S2[1] = `"C"`. They do **not match**. We take the maximum of:
  - `dp[0][2]` (which is 0) and
  - `dp[1][1]` (which we just computed as 1). So, `dp[1][2] = max(0, 1) = 1`.

The table now:

	""	A	C
""	0	0	0

	""	A	C
A	0	1	1
B	0		
C	0		

Filling Row 2 (for S1's "B"):

- **Cell (2,1):** Compare S1[1] = `"B"` with S2[0] = `"A"`. They do **not match**. We take the maximum of:
  - `dp[1][1]` (which is 1) and
  - `dp[2][0]` (which is 0). So, `dp[2][1] = max(1, 0) = 1`.
- **Cell (2,2):** Compare S1[1] = `"B"` with S2[1] = `"C"`. They do **not match**. We consider:
  - `dp[1][2]` (which is 1) and
  - `dp[2][1]` (which is 1). So, `dp[2][2] = max(1, 1) = 1`.

The table now:

	""	A	C
""	0	0	0
A	0	1	1
B	0	1	1
C	0		

Filling Row 3 (for S1's "C"):

- **Cell (3,1):** Compare S1[2] = `"C"` with S2[0] = `"A"`. They do **not match**. We take the maximum of:
  - `dp[2][1]` (which is 1) and
  - `dp[3][0]` (which is 0). So, `dp[3][1] = max(1, 0) = 1`.
- **Cell (3,2):** Compare S1[2] = `"C"` with S2[1] = `"C"`. They **match**. Hence, we look to the diagonal top-left cell: `dp[2][1]` (which is 1), and add 1 for the match, giving: `dp[3][2] = dp[2][1] + 1 = 1 + 1 = 2`.

The fully filled table:

	""	A	C
""	0	0	0
A	0	1	1
B	0	1	1
C	0	1	2

Step 3. Tracing the Logic

- **Matching Case (Cell (3,2)):** At cell (3,2), we are comparing `"C"` (from S1) with `"C"` (fro m S2). Since they match, it means that whatever common sequence we found before (in t he subproblem that ends at cell (2,1)) can now be extended by this matching character. T hat's why we use `dp[3][2] = dp[2][1] + 1` which gives us **2**. In our example, the common sequence built so far (from `"A"` from row 1 and column 1) is `"A"`, and adding `"C"` now gives us `"AC"`.
- **Non-Matching Case (Cell (1,2)):** At cell (1,2), we compare `"A"` (from S1) with `"C"` (from S2) . Because they don't match, we must decide whether to ignore the character from S1 or fr om S2.
  - `dp[0][2]` corresponds to ignoring `"A"` (from S1) and
  - `dp[1][1]` corresponds to keeping `"A"` and ignoring `"C"` (from S2). The maximum of the se two is `max(0, 1) = 1`, which tells us that the best common subsequence up to th at point remains `"A"`.

This same logic is applied at every cell:

- **Matching:** Extend the subsequence by 1.
- **Non-matching:** Choose the best result from the subproblems that come from skipping a char acter in either string.

At the end, the value in the bottom-right cell, `dp[3][2] = 2`, tells us the length of the LCS is **2**. By following the path of matching, we know the L CS here is `"AC"`.

Visualization (Illustrated DP Table)

Copy

```
          S2:   ""    A    C
        ----------------------
S1:  ""  |   0 |   0 |   0 |
     A   |   0 |   1 |   1 |   <-- "A" matches A; then "A" vs "C" takes the best v
     B   |   0 |   1 |   1 |   <-- "B" doesn't match; best remains from previous c
     C   |   0 |   1 |   2 |   <-- "C" matches C; extend the sequence from the dia
```

Final Thoughts

This example shows how the DP table is built by considering each pair of characters from the two strings, deciding whether they contribute to the common subsequence (if they match) or whether we inherit the best result so far (if they do not match). The recurrence relations:

- **Match:** `dp[i][j] = dp[i-1][j-1] + 1`
- **No Match:** `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`

are both logical ways to combine smaller subproblem solutions into the solution for larger pr
oblems.